

Reproducing the Displayed FORCE-Learning Cases in Chaotic Recurrent Neural Networks

Yin Neiming

July 27, 2026

Abstract

This report audits an independent MATLAB reproduction of all displayed computational cases in Sussillo and Abbott (2009) [1]. The implementation covers the Figure 2 output family, Figures 3–5 diagnostics, the three feedback/internal-learning cases in Figure 6, the multifunction and memory tasks in Figure 7, the motion-capture task in Figure 8, and Supplementary Figures S1–S2. Each case uses the architecture shown in the paper; Figure 2D is additionally benchmarked with the Figure 1A, 1B, and 1C architectures. Exact numerical targets are distinguished from structural reconstructions when coefficients or waveforms were not published. At the available scaled network sizes, Figure 2D (1A and 1C), the square wave, internal learning in Figure 6C, four-bit memory, motion capture, PCA diagnostics, gain dependence, and the supplementary spectral effect are reproduced. Failed or partial scaled cases are retained and reported rather than discarded. Code, fixed seeds, tests, figures, CSV summaries, and intermediate MAT files are versioned together.

1 Background and Scope

Recurrent neural networks can produce rich spontaneous dynamics because activity is continuously fed back through recurrent synapses. When the recurrent gain is large, random rate networks can become chaotic [2]. This creates a useful reservoir of temporal activity, but it also makes direct control difficult.

Sussillo and Abbott [1] proposed FORCE learning as a way to turn chaotic spontaneous activity into coherent target patterns. The algorithm rapidly suppresses output error during learning and then leaves a stable set of weights that can autonomously regenerate the desired signal. The paper demonstrates many outputs, including periodic signals, noisy targets, discontinuous functions, Lorenz-like trajectories, and movement patterns.

The paper defines three architectures. Figure 1A learns a linear readout and feeds its output directly back to the generator. Figure 1B inserts a separate feedback network. Figure 1C modifies recurrent generator weights internally. This project uses the architecture attached to each displayed case in the paper. Only Figure 2D is deliberately repeated across all three architectures, as an architecture benchmark agreed before implementation.

The scope includes main-text Figures 2–8 and Supplementary Figures S1–S2. Panels that define an input protocol rather than a separate output task (Figure 7A and 7C) are reproduced as part of the corresponding Figure 7B and 7D simulations. A result is called *exact-target* only when the paper or supplemental code publishes the numerical target. Unpublished waveforms, coefficients, and implementation details are labelled structural reconstructions. Long-horizon phase error is not treated as equivalent to failure for a chaotic target, but it is still reported numerically.

2 Methods

2.1 Rate-network dynamics and architectures

For the generator state $x \in \mathbf{R}^N$ and rates $r = \tanh(x)$, the Figure 1A implementation uses

$$x(t + \Delta t) = (1 - \Delta t)x(t) + \Delta t M r(t) + \Delta t w_f z(t), \quad z(t) = w^\top r(t).$$

The sparse random recurrent matrix has connection probability p and nonzero scale $g/\sqrt{pN}$. In Figure 1A only w is learned. Figure 1B adds a feedback state y , rates $s = \tanh(y)$, generator-to-feedback weights J^{FG} , and feedback-to-generator weights J^{GF} . Figure 1C applies the readout-weight update to selected rows of M , so the desired pattern becomes internal network dynamics. The implementations are in `run_force_external.m`, `run_force_feedback_network.m`, and `run_force_internal_all2all.m`.

2.2 FORCE/RLS update

At a learning step the pre-update error is $e^- = w^\top r - f$. With $P(0) = I/\alpha$, the shared RLS update is

$$k = P r, \quad c = (1 + r^\top k)^{-1}, \quad P \leftarrow P - k k^\top c, \quad w \leftarrow w - e^- k c.$$

The update is isolated in `rls_update.m`. Training leaves feedback loops closed and unclamped. Testing continues from the terminal training state with all weights fixed. We report mean absolute error (MAE), root-mean-square error where relevant, correlation, final readout norm, and task-specific accuracy.

2.3 Targets and case assignment

The exact Figure 2D target follows the publisher-supplied MATLAB example:

$$f(t) = \frac{a}{1.5} \left[\sin(\pi qt) + \frac{1}{2} \sin(2\pi qt) + \frac{1}{6} \sin(3\pi qt) + \frac{1}{3} \sin(4\pi qt) \right],$$

with $a = 1.3$ and $q = 1/60$. Figure 2H integrates the Lorenz system with $(\sigma, \beta, \rho) = (10, 8/3, 28)$. The triangle sequence, four-sine target, noisy-target protocol, Lorenz parameters, and fast/slow sine periods are numerical reconstructions from the paper or supplement. Figure 2E, 2G, and 2J use deterministic structural waveforms because complete numerical specifications were not published.

Table 1: Architecture assignment for the displayed cases. Page references are to the supplied main-paper PDF.

Case	Architecture	Implemented purpose
Figure 2A–K	1A	Output repertoire; 2D also repeated with 1B and 1C
Figure 3	1A	PCA of rates and readout-weight trajectory
Figure 4	1A	FORCE/output feedback versus target-clamped feedback
Figure 5	1A	Recurrent-gain sweep
Figure 6A	1A	Delayed nonlinear output feedback
Figure 6B	1B	Separate feedback network
Figure 6C	1C	Internal recurrent learning
Figure 7B, 7D	1C	Control-selected patterns and four-bit memory
Figure 8	1B	Running/walking motion channels
Supplement S1, S2	1A	Scalar-rate delta rule and effective spectra

2.4 Scaled experiments and data provenance

The exact reference-scale Figure 2D run uses $N = 1000$, $p = 0.1$, $g = 1.5$, $\alpha = 1$, $\Delta t = 0.1$, and 1440 simulated time units. Expensive multi-trial or multi-network cases use $N = 300$ – 500 , except Figure 6C ($N = 750$). These substitutions are recorded in each MAT-file configuration and are central to interpreting failures. Figure 8 uses CMU Graphics Lab trials 09_02.amc (running) and 08_01.amc (walking): root translation is removed, angles are unwrapped, smoothed with a five-frame moving average, interpolated tenfold, demeaned, and reduced/padded to 95 channels. The source permits research and commercial use but requests acknowledgment and prohibits direct resale [3].

2.5 Reproducibility controls

Every stochastic run fixes a MATLAB Twister seed. Unit tests cover target dimensions, RLS dimensions and symmetry, fixed-seed repeatability, stored-rate dimensions, AMC parsing, and finite non-RLS learning rates. Generated results are saved under `results/data`; publication-ready PNG and vector PDF figures are saved under `results/figures`. Zotero item D2YQS7LK was exported to `report/bibliography/zotero.bib`; its citation key is `sussillo_generating_2009`.

3 Results

3.1 Case-by-case audit

Table 2 reports the stored outputs without seed selection. “Success” means that the scaled implementation supports the displayed claim; “partial” means that training or qualitative structure is present but autonomous accuracy is weaker; “scaled failure” means that the available smaller network did not stabilize. Figure 2K is an intended failure case in the paper, so reproducing uncontrolled low-amplitude output is the expected result.

Table 2: Audit of main-text and supplementary cases.

Case	Arch.	Stored quantitative evidence	Assessment
Fig. 2A–C	1A	test MAE 0.289; correlation 0.833	Partial triangle-sequence reconstruction
Fig. 2D	1A	train MAE 0.00282; test MAE 0.0403; correlation 0.996	Success; exact published target
Fig. 2D	1B	test MAE 1.073; correlation -0.100	Scaled failure ($N = 500$, $N_F = 95$)
Fig. 2D	1C	test MAE 0.0729; correlation 0.926	Success at internal-learning scale
Fig. 2E	1A	test MAE 0.547; correlation 0.045	Scaled failure; unpublished 16 coefficients
Fig. 2F	1A	noisy-target test MAE 0.419; correlation 0.612	Partial noise-filtering behavior
Fig. 2G	1A	test MAE 0.0447; correlation 0.966	Success; structural square wave
Fig. 2H	1A	train MAE 0.0277; long test MAE 0.881	Training fit; chaotic trajectories diverge
Fig. 2I fast	1A	test MAE 0.222; correlation 0.941	Partial 60-ms-period reconstruction
Fig. 2I slow	1A	test MAE 2.139; correlation 0.055	Scaled failure for 8-s period
Fig. 2J	1A, two loops	test MAE 0.160; correlation 0.827	Partial one-shot structural reconstruction
Fig. 2K	1A	correlation 0.027 at amplitude 0.05	Expected chaos-control failure reproduced
Fig. 3	1A	eight-PC output reconstruction and long-tail spectrum	Qualitative PCA reproduction
Fig. 4	1A	stable trials fall from 80% at $\gamma = 0$ to 0% for $\gamma \geq 0.2$	Feedback-mismatch trend reproduced
Fig. 5	1A	RMSE falls to 0.078 and $\ w\ _2$ to 0.355 at $g = 1.55$	Gain dependence reproduced
Fig. 6A	1A	test MAE 1.443	Scaled delayed-feedback failure
Fig. 6B	1B	test MAE 0.948	Scaled separate-network failure
Fig. 6C	1C	test MAE 0.00550	Success
Fig. 7B	1C	mean five-pattern MAE 0.519	Scaled multifunction failure
Fig. 7D	1C	four-bit sign accuracy 98.74%	Success
Fig. 8	1B	run/walk correlations 0.920/0.968	Success on 95 motion channels
Fig. S1	1A	train-tail MAE 0.00605; frozen test MAE 0.0612	Training follows; scaled autonomous failure
Fig. S2	1A	maximum spectral shift 0.984 ($g = .8$), 0.408 ($g = 1.5$)	Qualitative spectral contrast reproduced

3.2 Output generation and architecture benchmark

Figure 1 compares the exact four-sine target across the three architectures. Figure 1A gives the strongest autonomous result. Figure 1C also stabilizes the target. The scaled Figure 1B implementation does not: the paper used a generator with 20,000 units, so reducing it to 500 units changes both representational capacity and feedback-network statistics. This negative result is not evidence against Figure 1B at the published scale.

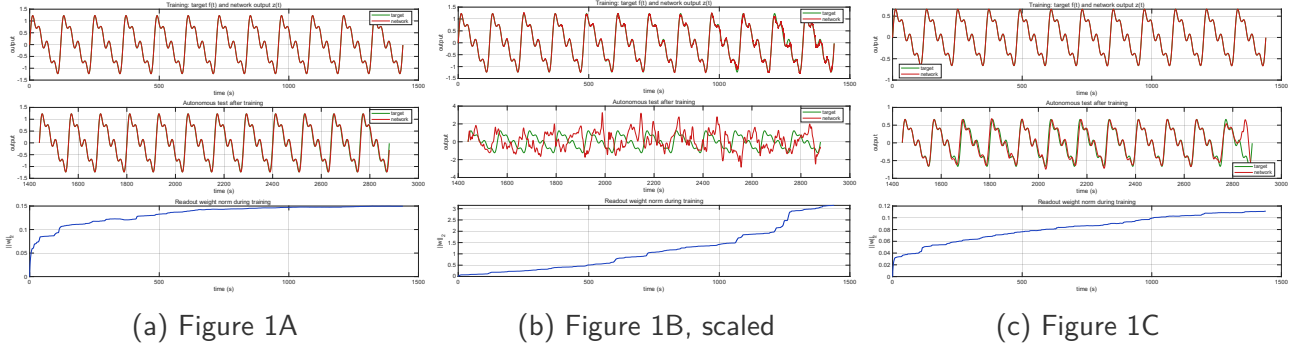


Figure 1: Figure 2D target under the three Figure 1 architectures. Source: main paper pp. 3–6 and publisher MATLAB examples.

The square wave is the strongest structural reconstruction (correlation 0.966). The unpublished 16-component target and the very slow sine do not stabilize at $N = 300$. The Lorenz readout fits during training, but pointwise long-horizon error grows because nearby chaotic trajectories separate; distributional or short-horizon metrics would be more appropriate than a single long correlation. All individual Figure 2 traces are retained in `results/figures`.

3.3 Dynamical diagnostics

The Figure 3 analysis stores network rates and readout history, computes an SVD of centered rates, and reconstructs the output from the leading eight components. The leading components recover the output while the eigenvalue spectrum retains a long tail, matching the paper’s qualitative account (main paper Figure 3).

The Figure 4 interpolation uses $\gamma z_{\text{target}} + (1 - \gamma)z$ during training, but always closes the loop with z during testing. As the train/test feedback mismatch increases, stability falls. In the paired-seed sweep, 80% of trials pass the fixed stability criterion at $\gamma = 0$, compared with none for $\gamma \geq 0.2$. Figure 5 shows the complementary gain effect: low- g networks require large weights and do not meet the cycle-MAE threshold, while $g \geq 1.25$ reaches it in one cycle; test RMSE falls sharply at the largest gains tested.

3.4 Feedback, control, memory, and motion

Of the Figure 6 variants, internal learning is robust at $N = 750$ (test MAE 0.00550), whereas delayed nonlinear feedback and the separately scaled feedback network fail. Figure 7B likewise needs the paper’s larger, pattern-partitioned network to reach low error; at $N = 300$ its mean MAE is 0.519. Figure 7D is much more robust: four maintained signs are read correctly on 98.74% of test samples after ON/OFF pulses.

The motion-capture experiment uses the paper’s named CMU trials and a scaled Figure 1B network with 95 outputs. Correlations of 0.920 (running) and 0.968 (walking) show that the first eight plotted

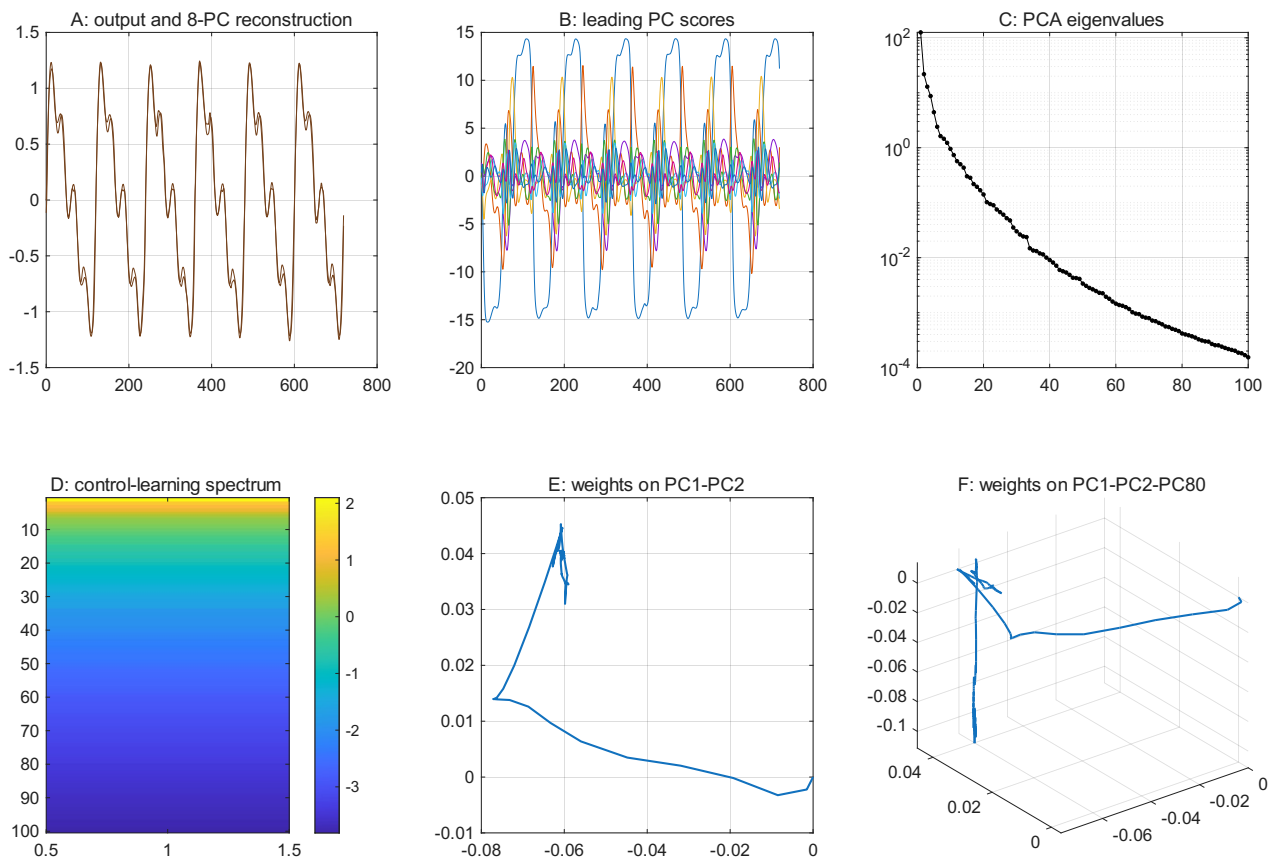


Figure 2: PCA diagnostics for the four-sine task. Source: main paper Figure 3, approximately pp. 6–7.

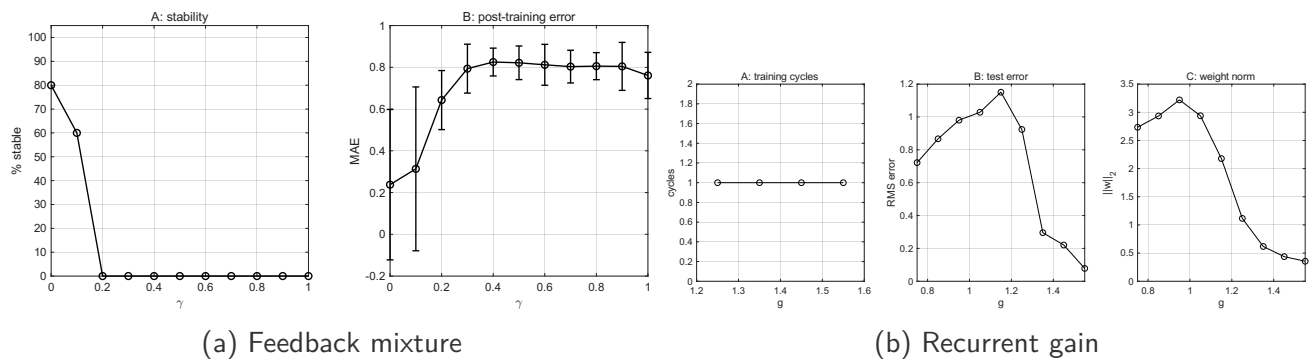


Figure 3: Figures 4 and 5 scaled diagnostics. Each parameter comparison uses paired random seeds. Source: main paper Figures 4–5.

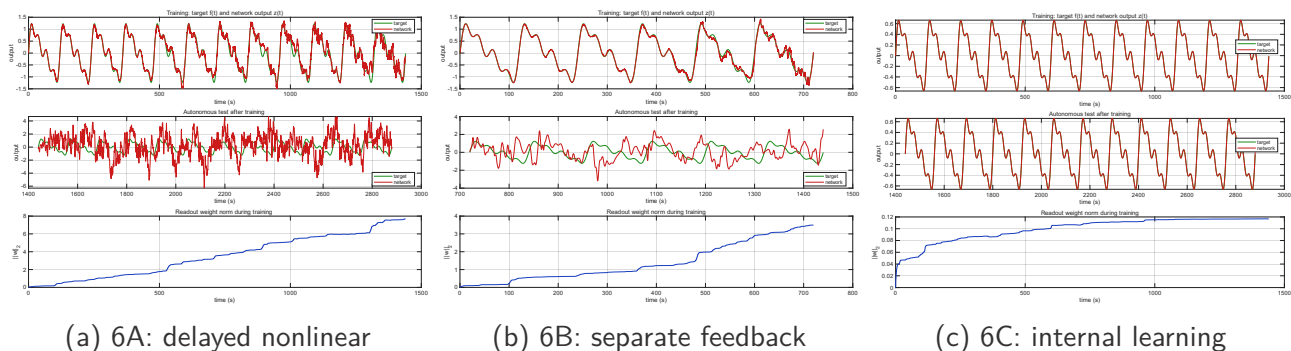


Figure 4: Figure 6 variants. Source: main paper Figure 6, approximately pp. 9–10.

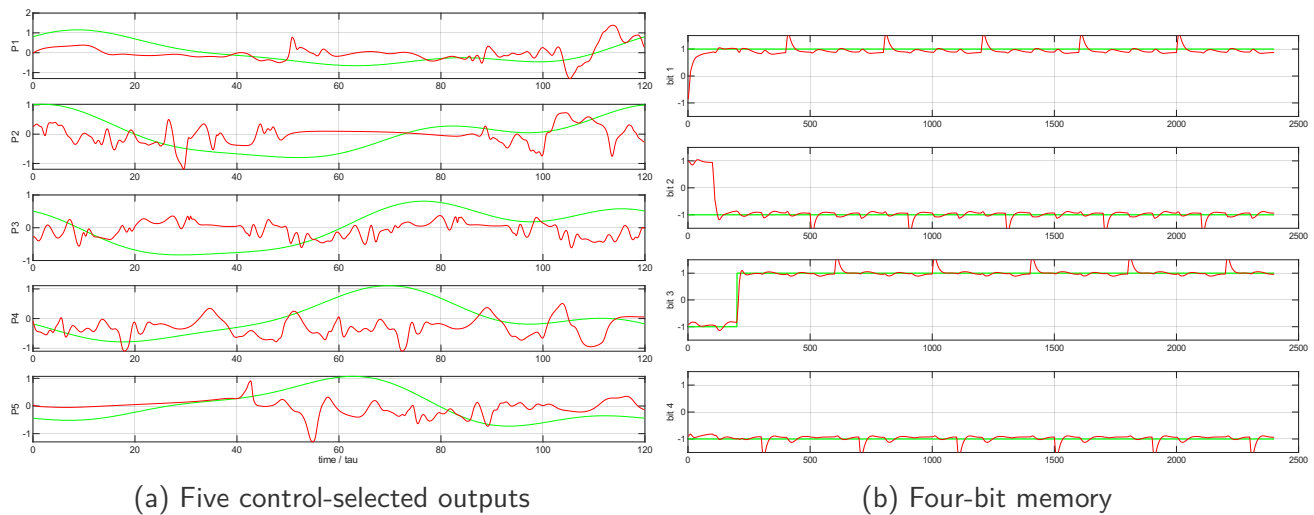


Figure 5: Figure 7 internal-learning tasks. Panels 7A and 7C define the control protocols represented in these simulations; panels 7B and 7D show the outputs.

channels and the aggregate 95-channel trajectories are tracked well, although this is a channel-level reconstruction rather than a rendered skeleton animation.

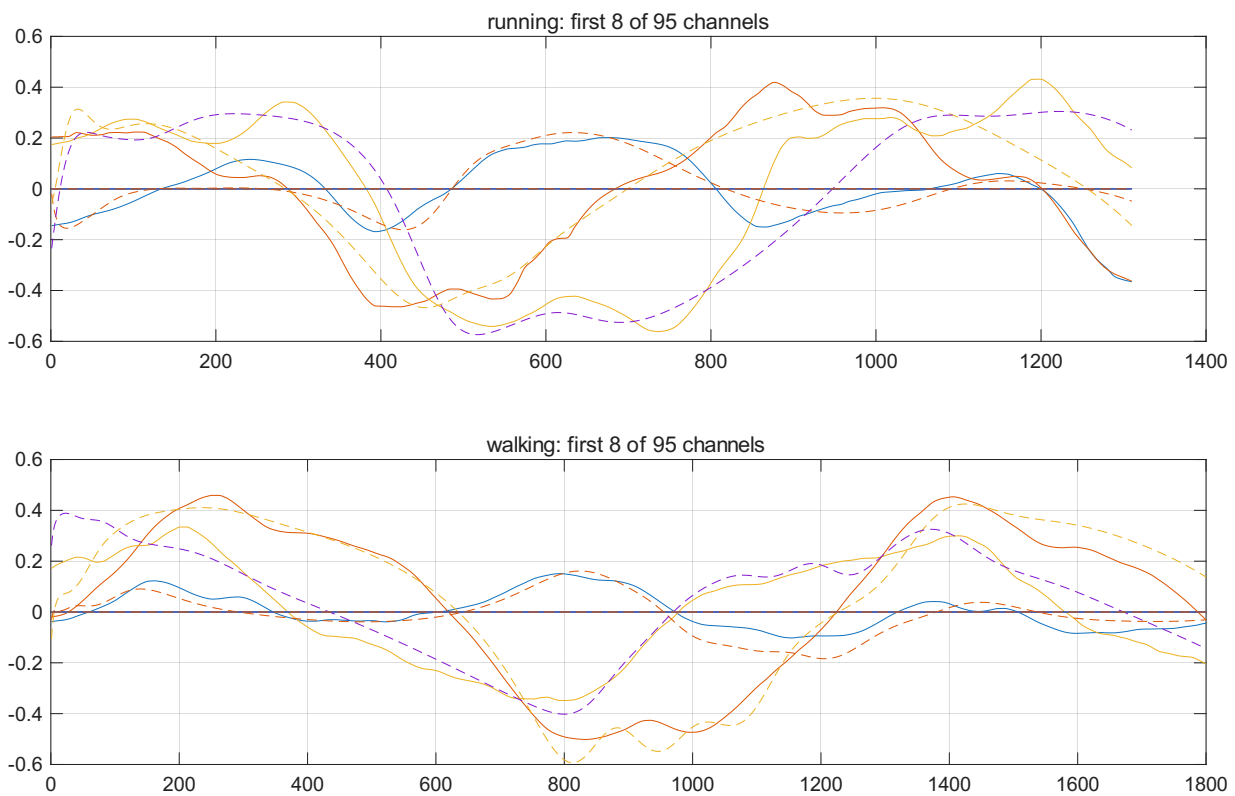


Figure 6: Running and walking targets (solid) and outputs (dashed), first 8 of 95 channels. Source: main paper Figure 8 and CMU trials 09_02/08_01.

3.5 Supplementary analyses

Supplementary Figure S1 replaces RLS by $w(t) = w(t - \Delta t) - \eta(t)e^-(t)r(t)$ and $\dot{\eta} = \eta[-\eta + |e|^{3/2}]$ in normalized units. The full reported duration of $60,000\tau$ is used, but at $N = 300$ the output follows only while learning remains active; frozen weights drift. Supplementary Figure S2 is reproduced more convincingly: at $g = 0.8$ a few effective eigenvalues move far from the pre-training disk, whereas at $g = 1.5$ the two spectra mostly overlap.

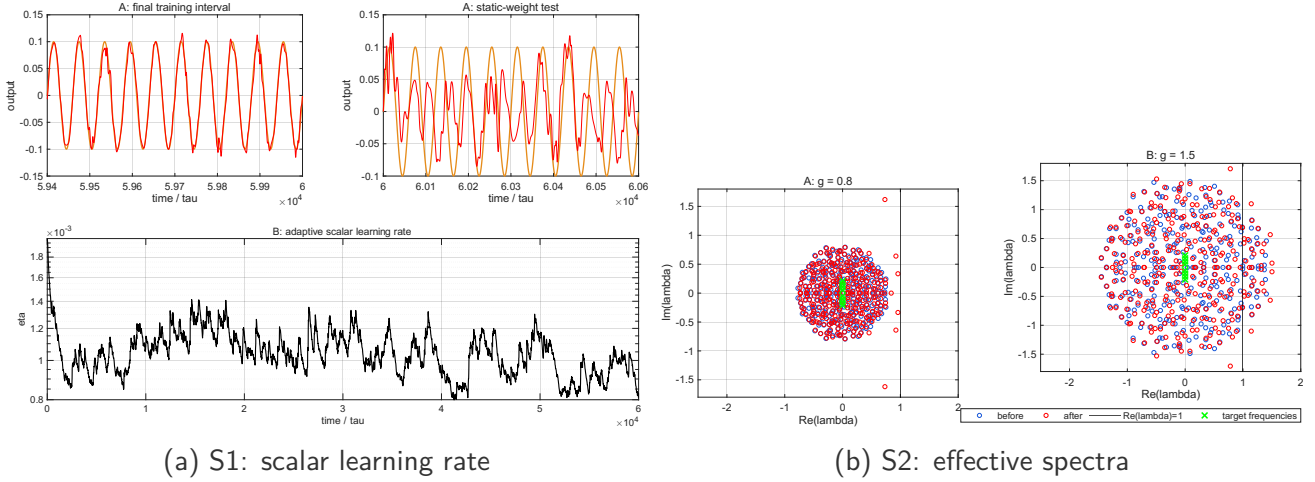


Figure 7: Supplementary cases. Green crosses in S2 mark the four target angular frequencies and the vertical line marks $\text{Re}(\lambda) = 1$.

4 Conclusion and Improvements

This reproduction supports the central claim that RLS can rapidly turn chaotic recurrent activity into coherent closed-loop output. The strongest evidence is the exact Figure 2D Figure 1A run (test correlation 0.996), the Figure 1C internal-learning run, the gain-dependent reduction in error and weight norm, the 98.74% four-bit memory accuracy, and the motion-channel correlations above 0.92. The effective-spectrum analysis also recovers the paper’s qualitative distinction between low and high recurrent gain.

The project also identifies a clear computational boundary. Cases that were published with very large generator or feedback networks do not necessarily survive reductions to a few hundred units. Figure 1B four-sine learning, delayed nonlinear feedback, the five-function internal task, the slow sine, and non-RLS autonomous output remain incomplete at the tested scales. These failures are useful results because they separate an algorithmic implementation from a claim of numerical replication.

The next improvements should be made in this order:

1. Run Figure 6B, Figure 7B, Figure 8, and S1 at the paper’s network sizes on a machine with sufficient RAM, recording wall time and peak memory.
2. Replace reconstructed Figure 2E/2J waveforms if original numerical target files become available; retain checksums with the data.
3. Evaluate Lorenz output with short-horizon error, attractor occupancy, power spectra, and Lyapunov-sensitive statistics instead of only long pointwise MAE.

4. Add parameter recovery sweeps around α , training duration, and feedback sparsity for failed cases, with paired seeds and confidence intervals.
5. Render the 95-channel Figure 8 result through the CMU skeleton hierarchy so motion quality can be judged geometrically, not only channel-wise.
6. Store environment metadata (MATLAB release, BLAS, CPU, RAM, TeX Live version) automatically with each full run.

The repository therefore distinguishes three outcomes: exact quantitative success, qualitative or partial reconstruction, and scaled failure. That distinction is necessary for an auditable paper reproduction and is more informative than presenting only the best-looking traces.

References

- [1] Sussillo, D. and Abbott, L. F. (2009). Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4), 544–557. doi:10.1016/j.neuron.2009.07.018. Metadata exported from local Zotero item D2YQS7LK.
- [2] Sompolinsky, H., Crisanti, A., and Sommers, H. J. (1988). Chaos in random neural networks. *Physical Review Letters*, 61(3), 259–262.
- [3] Carnegie Mellon University Graphics Lab. *Motion Capture Database*. <https://mocap.cs.cmu.edu/>. Trials 09_02 and 08_01; accessed July 2026.

A Reproducibility Map

Artifact	Purpose
scripts/run_figure2_suite.m	Figure 2 cases and Figure 2D architecture benchmark
scripts/run_figures3_to5.m	PCA, feedback interpolation, and gain sweep
scripts/run_figures6_to8.m	feedback/internal tasks, memory, and motion capture
scripts/run_supplement.m	Supplementary Figures S1 and S2
tests/run_all_tests.m	unit and smoke integration tests
results/data/*.csv	compact metrics used in this report
results/data/*.mat	configurations, traces, weights, and intermediate results
docs/workflow.md	Chinese teaching-style derivation and runbook

A.1 Recommended execution order

Run the tests first, then execute each case group independently:

```
matlab -batch "run('tests/run_all_tests.m')"  
matlab -batch "run('scripts/run_figure2_suite.m')"  
matlab -batch "run('scripts/run_figures3_to5.m')"  
matlab -batch "run('scripts/run_figures6_to8.m')"  
matlab -batch "run('scripts/run_supplement.m')"
```

The full groups overwrite their own deterministic outputs. Compile this report from report/ with latexmk -pdf main.tex. The generated report includes figures directly from results/figures, so no manual copying is required.

A.2 Versioned milestones

The implementation was committed one case group at a time: reusable external FORCE core (d3be30d), Figure 2 suite (39155a0), Figures 3–5 (b0c9f56), Figures 6–8 (7895f5c), and Supplementary Figures S1–S2 (d4c2527). These hashes identify the code and generated evidence used by the report.